CHAPTER

TWENTYFIVE

NETWORK PROTOCOLS - MANAGED NETWORK

25.1 EFI Managed Network Protocol

This chapter defines the EFI Managed Network Protocol. It is split into the following two main sections:

- Managed Network Service Binding Protocol (MNSBP)
- Managed Network Protocol (MNP)

The MNP provides raw (unformatted) asynchronous network packet I/O services. These services make it possible for multiple-event-driven drivers and applications to access and use the system network interfaces at the same time.

25.1.1 EFI_MANAGED_NETWORK_SERVICE_BINDING_PROTOCOL

Summary

The MNSBP is used to locate communication devices that are supported by an MNP driver and to create and destroy instances of the MNP child protocol driver that can use the underlying communications device.

The EFI Service Binding Protocol in *EFI Services Binding* defines the generic Service Binding Protocol functions. This section discusses the details that are specific to the MNP.

GUID

```
#define EFI_MANAGED_NETWORK_SERVICE_BINDING_PROTOCOL_GUID \
  {0xf36ff770,0xa7e1,0x42cf,\
    {0x9e,0xd2,0x56,0xf0,0xf2,0x71,0xf4,0x4c}}
```

Description

A network application (or driver) that requires shared network access can use one of the protocol handler services, such as *BS->LocateHandleBuffer()*, to search for devices that publish an MNSBP GUID. Each device with a published MNSBP GUID supports MNP and may be available for use.

After a successful call to the *EFI_MANAGED_NETWORK_SERVICE_BINDING_PROTOCOL.CreateChild()* function, the child MNP driver instance is in an unconfigured state; it is not ready to send and receive data packets.

Before a network application terminates execution, every successful call to the EFI_MANAGED_NETWORK_SERVICE_BINDING_PROTOCOL.CreateChild() function must be matched with a call to the EFI_MANAGED_NETWORK_SERVICE_BINDING_PROTOCOL.DestroyChild() function.

25.1.2 EFI_MANAGED_NETWORK_PROTOCOL

Summary

The MNP is used by network applications (and drivers) to perform raw (unformatted) asynchronous network packet I/O.

GUID

```
#define EFI_MANAGED_NETWORK_PROTOCOL_GUID\
  {0x7ab33a91, 0xace5, 0x4326,\
    {0xb5, 0x72, 0xe7, 0xee, 0x33, 0xd3, 0x9f, 0x16}}
```

Protocol Interface Structure

```
typedef struct \_EFI_MANAGED_NETWORK_PROTOCOL {
  EFI_MANAGED_NETWORK_GET_MODE_DATA        GetModeData;
  EFI_MANAGED_NETWORK_CONFIGURE            Configure;
  EFI_MANAGED_NETWORK_MCAST_IP_TO_MAC      McastIpToMac;
  EFI_MANAGED_NETWORK_GROUPS               Groups;
  EFI_MANAGED_NETWORK_TRANSMIT             Transmit;
  EFI_MANAGED_NETWORK_RECEIVE              Receive;
  EFI_MANAGED_NETWORK_CANCEL               Cancel;
  EFI_MANAGED_NETWORK_POLL                 Poll;
}   EFI_MANAGED_NETWORK_PROTOCOL;
```

Parameters

GetModeData
Returns the current MNP child driver operational parameters. May also support returning underlying Simple Network Protocol (SNP) driver mode data. See the GetModeData() function description.

Configure
Sets the *Configure()* function description.

McastIpToMac
Translates a software (IP) multicast address to a hardware (MAC) multicast address. This function may be unsupported in some MNP implementations. See the *McastIpToMac()* function description.

Groups
Enables and disables receive filters for multicast addresses. This function may be unsupported in some MNP implementations. See the *Groups()* function description.

Transmit
Places asynchronous outgoing data packets into the transmit queue. See the *Transmit()* function description.

Receive
Places an asynchronous receiving request into the receiving queue. See the *Receive()* function description.

Cancel
Aborts a pending transmit or receive request. See the *Cancel()* function description.

Poll
Polls for incoming data packets and processes outgoing data packets. See the *Poll()* function description.

Description

The services that are provided by MNP child drivers make it possible for multiple drivers and applications to send and receive network traffic using the same network device.

Before any network traffic can be sent or received, the *EFI_MANAGED_NETWORK_PROTOCOL.Configure()* function must initialize the operational parameters for the MNP child driver instance. Once configured, data packets can be received and sent using the following functions:

- EFI_MANAGED_NETWORK_PROTOCOL.Transmit()
- EFI_MANAGED_NETWORK_PROTOCOL.Receive()
- EFI_MANAGED_NETWORK_PROTOCOL.Poll()

25.1.3 EFI_MANAGED_NETWORK_PROTOCOL.GetModeData()

Summary

Returns the operational parameters for the current MNP child driver. May also support returning the underlying SNP driver mode data.

Prototype

```
typedef
EFI_STATUS
(EFIAPI *EFI_MANAGED_NETWORK_GET_MODE_DATA) (
  IN EFI_MANAGED_NETWORK_PROTOCOL               *This,
  OUT EFI_MANAGED_NETWORK_CONFIG_DATA           *MnpConfigData OPTIONAL,
  OUT EFI_SIMPLE_NETWORK_MODE                   *SnpModeData OPTIONAL
  );
```

Parameters

This
Pointer to the *EFI_MANAGED_NETWORK_PROTOCOL* instance.

MnpConfigData
Pointer to storage for MNP operational parameters. Type *EFI_MANAGED_NETWORK_CONFIG_DATA* is defined in "Related Definitions" below.

SnpModeData
Pointer to storage for SNP operational parameters. This feature may be unsupported. Type *EFI_SIMPLE_NETWORK_MODE* is defined in the *EFI_SIMPLE_NETWORK_PROTOCOL*.

Description

The *GetModeData()* function is used to read the current mode data (operational parameters) from the MNP or the underlying SNP.

Related Definitions

```
//*******************************************************
// EFI_MANAGED_NETWORK_CONFIG_DATA
//*******************************************************
typedef struct {
  UINT32         ReceivedQueueTimeoutValue;
  UINT32         TransmitQueueTimeoutValue;
  UINT16         ProtocolTypeFilter;
  BOOLEAN        EnableUnicastReceive;
  BOOLEAN        EnableMulticastReceive;
  BOOLEAN        EnableBroadcastReceive;
  BOOLEAN        EnablePromiscuousReceive;
```

(continues on next page)

(continued from previous page)

```
  BOOLEAN        FlushQueuesOnReset;
  BOOLEAN        EnableReceiveTimestamps;
  BOOLEAN        DisableBackgroundPolling;
}   EFI_MANAGED_NETWORK_CONFIG_DATA;
```

ReceivedQueueTimeoutValue
Timeout value for a UEFI one-shot timer event. A packet that has not been removed from the MNP receive queue by a call to *EFI_MANAGED_NETWORK_PROTOCOL.Poll()* will be dropped if its receive timeout expires. If this value is zero, then there is no receive queue timeout. If the receive queue fills up, then the device receive filters are disabled until there is room in the receive queue for more packets. The startup default value is 10,000,000 (10 seconds).

TransmitQueueTimeoutValue
Timeout value for a UEFI one-shot timer event. A packet that has not been removed from the MNP transmit queue by a call to *EFI_MANAGED_NETWORK_PROTOCOL.Poll()* will be dropped if its transmit timeout expires. If this value is zero, then there is no transmit queue timeout. If the transmit queue fills up, then the *EFI_MANAGED_NETWORK_PROTOCOL.Transmit()* function will return *EFI_NOT_READY* until there is room in the transmit queue for more packets. The startup default value is 10,000,000 (10 seconds).

ProtocolTypeFilter
Ethernet type II 16-bit protocol type in host byte order. Valid values are zero and 1,500 to 65,535. Set to zero to receive packets with any protocol type. The startup default value is zero.

EnableUnicastReceive
Set to *TRUE* to receive packets that are sent to the network device MAC address. The startup default value is *FALSE*.

EnableMulticastReceive
Set to *TRUE* to receive packets that are sent to any of the active multicast groups. The startup default value is *FALSE*.

EnableBroadcastReceive
Set to *TRUE* to receive packets that are sent to the network device broadcast address. The startup default value is *FALSE* .

EnablePromiscuousReceive
Set to *TRUE* to receive packets that are sent to any MAC address. Note that setting this field to *TRUE* may cause packet loss and degrade system performance on busy networks. The startup default value is *FALSE*.

FlushQueuesOnReset
Set to *TRUE* to drop queued packets when the configuration is changed. The startup default value is *FALSE*.

EnableReceiveTimestamps
Set to *TRUE* to timestamp all packets when they are received by the MNP. Note that timestamps may be unsupported in some MNP implementations. The startup default value is *FALSE*.

DisableBackgroundPolling
Set to *TRUE* to disable background polling in this MNP instance. Note that background polling may not be supported in all MNP implementations. The startup default value is *FALSE*, unless background polling is not supported.

Status Codes Returned

EFI_SUCCESS	The operation completed successfully.
EFI_INVALID_PARAMETER	*This* is *NULL*.
EFI_UNSUPPORTED	The requested feature is unsupported in this MNP implementation.

continues on next page

Table 25.1 – continued from previous page

EFI_NOT_STARTED	This MNP child driver instance has not been configured. The default values are returned in *MnpConfigData* if it is not *NULL*.
Other	The mode data could not be read.

25.1.4 EFI_MANAGED_NETWORK_PROTOCOL.Configure()

Summary

Sets or clears the operational parameters for the MNP child driver.

Prototype

```
typedef
EFI_STATUS
(EFIAPI *EFI_MANAGED_NETWORK_CONFIGURE) (
  IN EFI_MANAGED_NETWORK_PROTOCOL        *This,
  IN EFI_MANAGED_NETWORK_CONFIG_DATA     *MnpConfigData OPTIONAL
  );
```

Parameters

This
Pointer to the *EFI_MANAGED_NETWORK_PROTOCOL* instance.

MnpConfigData
Pointer to configuration data that will be assigned to the MNP child driver instance. If *NULL*, the MNP child driver instance is reset to startup defaults and all pending transmit and receive requests are flushed. Type *EFI_MANAGED_NETWORK_CONFIG_DATA* is defined in *EFI_MANAGED_NETWORK_PROTOCOL.GetModeData()*.

Description

The *Configure()* function is used to set, change, or reset the operational parameters for the MNP child driver instance. Until the operational parameters have been set, no network traffic can be sent or received by this MNP child driver instance. Once the operational parameters have been reset, no more traffic can be sent or received until the operational parameters have been set again.

Each MNP child driver instance can be started and stopped independently of each other by setting or resetting their receive filter settings with the *Configure()* function.

After any successful call to *Configure()*, the MNP child driver instance is started. The internal periodic timer (if supported) is enabled. Data can be transmitted and may be received if the receive filters have also been enabled.

NOTE: *If multiple MNP child driver instances will receive the same packet because of overlapping receive filter settings, then the first MNP child driver instance will receive the original packet and additional instances will receive copies of the original packet.*

NOTE: WARNING: *Receive filter settings that overlap will consume extra processor and/or DMA resources and degrade system and network performance.*

Status Codes Returned

EFI_SUCCESS	The operation completed successfully.

continues on next page

Table 25.2 – continued from previous page

EFI_INVALID_PARAMETER	One or more of the following conditions is *TRUE* : • *This* is *NULL*. • *MnpConfigData.ProtocolTypeFilter* is not valid. • The operational data for the MNP child driver instance is unchanged.
EFI_OUT_OF_RESOURCES	Required system resources (usually memory) could not be allocated. The MNP child driver instance has been reset to startup defaults.
EFI_UNSUPPORTED	The requested feature is unsupported in this [MNP] implementation. The operational data for the MNP child driver instance is unchanged.
EFI_DEVICE_ERROR	An unexpected network or system error occurred. The MNP child driver instance has been reset to startup defaults.
Other	The MNP child driver instance has been reset to startup defaults.

25.1.5 EFI_MANAGED_NETWORK_PROTOCOL.McastIpToMac()

Summary

Translates an IP multicast address to a hardware (MAC) multicast address. This function may be unsupported in some MNP implementations.

Prototype

```
typedef
EFI_STATUS
(EFIAPI *EFI_MANAGED_NETWORK_MCAST_IP_TO_MAC) (
  IN EFI_MANAGED_NETWORK_PROTOCOL         *This,
  IN BOOLEAN                              Ipv6Flag,
  IN EFI_IP_ADDRESS                       *IpAddress,
  OUT EFI_MAC_ADDRESS                     *MacAddress
  );
```

Parameters

This
Pointer to the *EFI_MANAGED_NETWORK_PROTOCOL* instance.

Ipv6Flag
Set to *TRUE* to if *IpAddress* is an IPv6 multicast address.

Set to *FALSE* if *IpAddress* is an IPv4 multicast address.

IpAddress
Pointer to the multicast IP address (in network byte order) to convert.

MacAddress
Pointer to the resulting multicast MAC address.

Description

The *McastIpToMac()* function translates an IP multicast address to a hardware (MAC) multicast address.

This function may be implemented by calling the underlying *EFI_SIMPLE_NETWORK.MCastIpToMac()* function, which may also be unsupported in some MNP implementations.

Status Codes Returned

EFI_SUCCESS	The operation completed successfully.
EFI_INVALID_PARAMETER	One of the following conditions is *TRUE* : • *This* is *NULL*. • *IpAddress* is *NULL*. • * *IpAddress* is not a valid multicast IP address. • *MacAddress* is *NULL*.
EFI_NOT_STARTED	This MNP child driver instance has not been configured.
EFI_UNSUPPORTED	The requested feature is unsupported in this MNP implementation.
EFI_DEVICE_ERROR	An unexpected network or system error occurred.
Other	The address could not be converted.

25.1.6 EFI_MANAGED_NETWORK_PROTOCOL.Groups()

Summary

Enables and disables receive filters for multicast address. This function may be unsupported in some MNP implementations.

Prototype

```
typedef
EFI_STATUS
(EFIAPI *EFI_MANAGED_NETWORK_GROUPS) (
  IN EFI_MANAGED_NETWORK_PROTOCOL      *This,
  IN BOOLEAN                           JoinFlag,
  IN EFI_MAC_ADDRESS                   *MacAddress OPTIONAL
  );
```

Parameters

This
Pointer to the *EFI_MANAGED_NETWORK_PROTOCOL* instance.

JoinFlag

Set to *TRUE* to join this multicast group.

Set to *FALSE* to leave this multicast group.

MacAddress
Pointer to the multicast MAC group (address) to join or leave.

Description

The *Groups()* function only adds and removes multicast MAC addresses from the filter list. The MNP driver does not transmit or process Internet Group Management Protocol (IGMP) packets.

If *JoinFlag* is *FALSE* and *MacAddress* is *NULL*, then all joined groups are left.

Status Codes Returned

EFI_SUCCESS	The requested operation completed successfully.
EFI_INVALID_PARAMETER	One or more of the following conditions is *TRUE* : • *This* is *NULL*. • *JoinFlag* is *TRUE* and *MacAddress* is *NULL*. • * *MacAddress* is not a valid multicast MAC address. The MNP multicast group settings are unchanged.
EFI_NOT_STARTED	This MNP child driver instance has not been configured.
EFI_ALREADY_STARTED	The supplied multicast group is already joined.
EFI_NOT_FOUND	The supplied multicast group is not joined.
EFI_DEVICE_ERROR	An unexpected network or system error occurred. The MNP child driver instance has been reset to startup defaults.
EFI_UNSUPPORTED	The requested feature is unsupported in this MNP implementation.
Other	The requested operation could not be completed. The MNP multicast group settings are unchanged.

25.1.7 EFI_MANAGED_NETWORK_PROTOCOL.Transmit()

Summary

Places asynchronous outgoing data packets into the transmit queue.

Prototype

```
typedef
EFI_STATUS
(EFIAPI *EFI_MANAGED_NETWORK_TRANSMIT) (
  IN EFI_MANAGED_NETWORK_PROTOCOL            *This,
  IN EFI_MANAGED_NETWORK_COMPLETION_TOKEN    *Token
  );
```

Parameters

This
Pointer to the *EFI_MANAGED_NETWORK_PROTOCOL* instance.

Token
Pointer to a token associated with the transmit data descriptor. Type *EFI_MANAGED_NETWORK_COMPLETION_TOKEN* is defined in "Related Definitions" below.

Description

The *Transmit()* function places a completion token into the transmit packet queue. This function is always asynchronous.

The caller must fill in the *Token.Event* and *Token.TxData* fields in the completion token, and these fields cannot be *NULL*. When the transmit operation completes, the MNP updates the *Token.Status* field and the *Token.Event* is signaled.

NOTE *There may be a performance penalty if the packet needs to be defragmented before it can be transmitted by the network device. Systems in which performance is critical should review the requirements and features of the underlying communications device and drivers*.

Related Definitions

```
//*******************************************************
// EFI_MANAGED_NETWORK_COMPLETION_TOKEN
//*******************************************************
typedef struct {
  EFI_EVENT                            Event;
  EFI_STATUS                           Status;
  union {
    EFI_MANAGED_NETWORK_RECEIVE_DATA   *RxData;
    EFI_MANAGED_NETWORK_TRANSMIT_DATA  *TxData;
  }                                    Packet;
}  EFI_MANAGED_NETWORK_COMPLETION_TOKEN;
```

Event

This *Event* will be signaled after the *Status* field is updated by the MNP. The type of *Event* must be *EVT_NOTIFY_SIGNAL*. The Task Priority Level (TPL) of *Event* must be lower than or equal to TPL_CALLBACK.

Status

This field will be set to one of the following values:

EFI_SUCCESS: The receive or transmit completed successfully.

EFI_ABORTED: The receive or transmit was aborted.

EFI_TIMEOUT: The transmit timeout expired.

EFI_DEVICE_ERROR: There was an unexpected system or network error.

EFI_NO_MEDIA: There was a media error

RxData

When this token is used for receiving, *RxData* is a pointer to the *EFI_MANAGED_NETWORK_RECEIVE_DATA*.

TxData

When this token is used for transmitting, *TxData* is a pointer to the *EFI_MANAGED_NETWORK_TRANSMIT_DATA*.

The *EFI_MANAGED_NETWORK_COMPLETION_TOKEN* structure is used for both transmit and receive operations.

When it is used for transmitting, the *Event* and *TxData* fields must be filled in by the MNP client. After the transmit operation completes, the MNP updates the *Status* field and the *Event* is signaled.

When it is used for receiving, only the *Event* field must be filled in by the MNP client. After a packet is received, the MNP fills in the *RxData* and *Status* fields and the *Event* is signaled.

```
//*******************************************************
// EFI_MANAGED_NETWORK_RECEIVE_DATA
//*******************************************************
typedef struct {
  EFI_TIME     Timestamp;
  EFI_EVENT    RecycleEvent;
```

(continues on next page)

(continued from previous page)

```
  UINT32       PacketLength;
  UINT32       HeaderLength;
  UINT32       AddressLength;
  UINT32       DataLength;
  BOOLEAN      roadcastFlag;
  BOOLEAN      MulticastFlag;
  BOOLEAN      PromiscuousFlag;
  UINT16       ProtocolType;
  VOID         *DestinationAddress;
  VOID         *SourceAddress;
  VOID         *MediaHeader;
  VOID         *PacketData;
}   EFI_MANAGED_NETWORK_RECEIVE_DATA;
```

Timestamp
System time when the MNP received the packet. *Timestamp* is zero filled if receive timestamps are disabled or unsupported.

RecycleEvent
MNP clients must signal this event after the received data has been processed so that the receive queue storage can be reclaimed. Once *RecycleEvent* is signaled, this structure and the received data that is pointed to by this structure must not be accessed by the client.

PacketLength
Length of the entire received packet (media header plus the data).

HeaderLength
Length of the media header in this packet.

AddressLength
Length of a MAC address in this packet.

DataLength
Length of the data in this packet.

BroadcastFlag
Set to *TRUE* if this packet was received through the broadcast filter. (The destination MAC address is the broadcast MAC address.)

MulticastFlag
Set to *TRUE* if this packet was received through the multicast filter. (The destination MAC address is in the multicast filter list.)

PromiscuousFlag
Set to *TRUE* if this packet was received through the promiscuous filter. (The destination address does not match any of the other hardware or software filter lists.)

ProtocolType
16-bit protocol type in host byte order. Zero if there is no protocol type field in the packet header.

DestinationAddress
Pointer to the destination address in the media header.

SourceAddress
Pointer to the source address in the media header.

MediaHeader
Pointer to the first byte of the media header.

PacketData
Pointer to the first byte of the packet data (immediately following media header).

An *EFI_MANAGED_NETWORK_RECEIVE_DATA* structure is filled in for each packet that is received by the MNP.

If multiple instances of this MNP driver can receive a packet, then the receive data structure and the received packet are duplicated for each instance of the MNP driver that can receive the packet.

```
//*******************************************************
// EFI_MANAGED_NETWORK_TRANSMIT_DATA
//*******************************************************
typedef struct {
  EFI_MAC_ADDRESS                    *DestinationAddress OPTIONAL;
  EFI_MAC_ADDRESS                    *SourceAddress OPTIONAL;
  UINT16                             ProtocolType OPTIONAL;
  UINT32                             DataLength;
  UINT16                             HeaderLength OPTIONAL;
  UINT16                             FragmentCount;
  EFI_MANAGED_NETWORK_FRAGMENT_DATA  FragmentTable[1];
}   EFI_MANAGED_NETWORK_TRANSMIT_DATA;
```

DestinationAddress
Pointer to the destination MAC address if the media header is not included in *FragmentTable[]*. If *NULL*, then the media header is already filled in *FragmentTable[]*.

SourceAddress
Pointer to the source MAC address if the media header is not included in *FragmentTable[]*. Ignored if *DestinationAddress* is *NULL*.

ProtocolType
The protocol type of the media header in host byte order. Ignored if *DestinationAddress* is *NULL*.

DataLength
Sum of all *FragmentLength* fields in *FragmentTable[]* minus the media header length.

HeaderLength
Length of the media header if it is included in the *FragmentTable*. Must be zero if *DestinationAddress* is not *NULL*.

FragmentCount
Number of data fragments in *FragmentTable[]*. This field cannot be zero.

FragmentTable
Table of data fragments to be transmitted. The first byte of the first entry in *FragmentTable[]* is also the first byte of the media header or, if there is no media header, the first byte of payload. Type *EFI_MANAGED_NETWORK_FRAGMENT_DATA* is defined below.

The *EFI_MANAGED_NETWORK_TRANSMIT_DATA* structure describes a (possibly fragmented) packet to be transmitted.

The *DataLength* field plus the *HeaderLength* field must be equal to the sum of all of the *FragmentLength* fields in the *FragmentTable*.

If the media header is included in *FragmentTable[]*, then it cannot be split between fragments.

```
//*******************************************************
// EFI_MANAGED_NETWORK_FRAGMENT_DATA
//*******************************************************
typedef struct {
```

(continues on next page)

(continued from previous page)

```
  UINT32       FragmentLength;
  VOID         *FragmentBuffer;
}   EFI_MANAGED_NETWORK_FRAGMENT_DATA;
```

FragmentLength
Number of bytes in the *FragmentBuffer*. This field may not be set to zero.

FragmentBuffer
Pointer to the fragment data. This field may not be set to *NULL*.

The *EFI_MANAGED_NETWORK_FRAGMENT_DATA* structure describes the location and length of a packet fragment to be transmitted.

Status Codes Returned

EFI_SUCCESS	The transmit completion token was cached.
EFI_NOT_STARTED	This MNP child driver instance has not been configured.
EFI_INVALID_PARAMETER	One or more of the following conditions is *TRUE* : • *This* is *NULL.* • *Token* is *NULL.* • *Token.Event* is *NULL.* • *Token.TxData is NULL.* • *Token.TxData.DestinationAddress is not NULL and Token.TxData.HeaderLength is zero.* • *Token.TxData.FragmentCount* is zero. • *(Token.TxData.HeaderLength + Token.TxData.DataLength)* is not equal to the sum of the *Token.TxDat a.FragmentTable[ ].FragmentLength* fields. • One or more of the *Token.TxDat a.FragmentTable[ ].FragmentLength* fields is zero. • One or more of the *Token.TxData.Frag mentTable[ ].FragmentBufferfields* is *NULL.* • *Token.TxData.DataLength is greater than MTU*
EFI_ACCESS_DENIED	The transmit completion token is already in the transmit queue.
EFI_OUT_OF_RESOURCES	The transmit data could not be queued due to a lack of system resources (usually memory).
EFI_DEVICE_ERROR	An unexpected system or network error occurred. The MNP child driver instance has been reset to startup defaults.
EFI_NOT_READY	The transmit request could not be queued because the transmit queue is full.
EFI_NO_MEDIA	There was a media error.

25.1.8 EFI_MANAGED_NETWORK_PROTOCOL.Receive()

Summary

Places an asynchronous receiving request into the receiving queue.

Prototype

```
typedef
EFI_STATUS
(EFIAPI *EFI_MANAGED_NETWORK_RECEIVE) (
  IN EFI_MANAGED_NETWORK_PROTOCOL            *This,
  IN EFI_MANAGED_NETWORK_COMPLETION_TOKEN    *Token
  );
```

Parameters

This
Pointer to the *EFI_MANAGED_NETWORK_PROTOCOL* instance.

Token

Pointer to a token associated with the receive data descriptor. Type *EFI_MANAGED_NETWORK_COMPLETION_TOKEN* is defined in *EFI_MANAGED_NETWORK_PROTOCOL.Transmit()* .

Description

The *Receive()* function places a completion token into the receive packet queue. This function is always asynchronous.

The caller must fill in the *Token.Event* field in the completion token, and this field cannot be *NULL*. When the receive operation completes, the MNP updates the *Token.Status* and *Token.RxData* fields and the *Token.Event* is signaled.

Status Codes Returned

EFI_SUCCESS	The receive completion token was cached.
EFI_NOT_STARTED	This MNP child driver instance has not been configured.
EFI_INVALID_PARAMETER	One or more of the following conditions is *TRUE* : • *This* is *NULL*. • *Token* is *NULL*. • *Token.Event* is *NULL*
EFI_OUT_OF_RESOURCES	The transmit data could not be queued due to a lack of system resources (usually memory).
EFI_DEVICE_ERROR	An unexpected system or network error occurred. The MNP child driver instance has been reset to startup defaults.
EFI_ACCESS_DENIED	The receive completion token was already in the receive queue.
EFI_NOT_READY	The receive request could not be queued because the receive queue is full.
EFI_NO_MEDIA	There was a media error.

25.1.9 EFI_MANAGED_NETWORK_PROTOCOL.Cancel()

Summary

Aborts an asynchronous transmit or receive request.

Prototype

```
typedef
EFI_STATUS
(EFIAPI *EFI_MANAGED_NETWORK_CANCEL)(
  IN EFI_MANAGED_NETWORK_PROTOCOL            *This,
  IN EFI_MANAGED_NETWORK_COMPLETION_TOKEN    *Token OPTIONAL
  );
```

Parameters

This
Pointer to the *EFI_MANAGED_NETWORK_PROTOCOL* instance.

Token

Pointer to a token that has been issued by *EFI_MANAGED_NETWORK_PROTOCOL.Transmit()* or *EFI_MANAGED_NETWORK_PROTOCOL.Receive()*. If *NULL*, all pending tokens are aborted. Type *EFI_MANAGED_NETWORK_COMPLETION_TOKEN* is defined in *EFI_MANAGED_NETWORK_PROTOCOL.Transmit()*.

Description

The *Cancel()* function is used to abort a pending transmit or receive request. If the token is in the transmit or receive request queues, after calling this function, *Token.Status* will be set to *EFI_ABORTED* and then *Token.Event* will be signaled. If the token is not in one of the queues, which usually means that the asynchronous operation has completed, this function will not signal the token and *EFI_NOT_FOUND* is returned.

Status Codes Returned

EFI_SUCCESS	The asynchronous I/O request was aborted and *Token.Event* was signaled. When *Token* is *NULL*, all pending requests were aborted and their events were signaled.
EFI_NOT_STARTED	This MNP child driver instance has not been configured.
EFI_INVALID_PARAMETER	*This* is *NULL*.
EFI_NOT_FOUND	When *Token* is not *NULL*, the asynchronous I/O request was not found in the transmit or receive queue. It has either completed or was not issued by *Transmit()* and *Receive()*.

25.1.10 EFI_MANAGED_NETWORK_PROTOCOL.Poll()

Summary

Polls for incoming data packets and processes outgoing data packets.

Prototype

```
typedef
EFI_STATUS
(EFIAPI *EFI_MANAGED_NETWORK_POLL) (
  IN EFI_MANAGED_NETWORK_PROTOCOL      *This
  );
```

Parameters

This
Pointer to the *EFI_MANAGED_NETWORK_PROTOCOL* instance.

Description

The *Poll()* function can be used by network drivers and applications to increase the rate that data packets are moved between the communications device and the transmit and receive queues.

Normally, a periodic timer event internally calls the *Poll()* function. But, in some systems, the periodic timer event may not call *Poll()* fast enough to transmit and/or receive all data packets without missing packets. Drivers and applications that are experiencing packet loss should try calling the *Poll()* function more often.

Status Codes Returned

EFI_SUCCESS	Incoming or outgoing data was processed.
EFI_NOT_STARTED	This MNP child driver instance has not been configured.
EFI_DEVICE_ERROR	An unexpected system or network error occurred. The MNP child driver instance has been reset to startup defaults.
EFI_NOT_READY	No incoming or outgoing data was processed. Consider increasing the polling rate.
EFI_TIMEOUT	Data was dropped out of the transmit and/or receive queue. Consider increasing the polling rate.